Anexo — Pruebas y cobertura del orquestador

Micro-SOAR · Plataforma móvil de respuesta a incidentes · Proyecto Final de Ingeniería

> Generado el 13 de octubre de 2026 a partir de la corrida real de la suite sobre el commit `2155de2`. Entorno: Node.js v24.19.0, Jest 29.7.0, base SQLite efímera. Regenerable con `docs/anexo-pruebas/generar.js`.

1. Resumen

49/49	11	98,9%	91,5%
tests aprobados	archivos de prueba	cobertura de líneas	cobertura de ramas

El orquestador (backend Node.js/Express que recibe las alertas de Wazuh, las normaliza en incidentes, las enriquece y ejecuta las acciones de respuesta) supera el umbral de **85%** exigido en las cuatro métricas de cobertura. El umbral no es solo un objetivo: está configurado en `coverageThreshold` y el pipeline de integración continua rechaza cualquier cambio que lo haga descender.

Las pruebas automatizadas se complementan con pruebas de punta a punta contra la infraestructura real en AWS (sección 5), que validan lo que los mocks no pueden: el comportamiento efectivo de Wazuh, del firewall del host y de la app en un dispositivo físico.

2. Estrategia de pruebas

Nivel	Qué valida	Cómo
Unitario	Normalización y filtrado de alertas; cliente de la API de Wazuh; enriquecimiento	Jest. Las llamadas HTTP externas (Wazuh, VirusTotal, AbuseIPDB) se reemplazan por mocks de `axios` que reproducen respuestas reales, incluidos los errores.
Integración (API)	Endpoints REST completos: webhook, incidentes, acciones de respuesta, autenticación, auditoría	Supertest contra la aplicación Express real y una base SQLite efímera creada con Prisma en cada corrida. Solo se aísla la frontera con Wazuh.
Punta a punta	Cadena completa ataque → detección → incidente → app → acción → efecto en la red	Manual y documentada, contra la infraestructura de AWS y un celular real por Tailscale (sección 5).

Criterios de diseño

- **Probar los modos de falla, no solo el camino feliz.** La mayoría de los defectos del proyecto fueron respuestas engañosas (Wazuh aceptando una acción que luego no ejecuta), por lo que cada acción de respuesta se prueba contra rechazo HTTP, rechazo con HTTP 200 y agente fallido.
- **Los servicios auxiliares no deben interrumpir el flujo crítico.** Se verifica que una caída de VirusTotal o de la auditoría no impida registrar un incidente ni iniciar sesión.
- **Sin dependencia del entorno local.** Las pruebas corren sin credenciales ni archivo `.env`; ninguna llega a la red. Así se ejecutan idénticas en la notebook de desarrollo y en GitHub Actions.
- **Datos de prueba realistas.** Las alertas simuladas (`orchestrator/mocks/`) replican alertas reales de Wazuh: fuerza bruta SSH (regla 5712), malware detectado por VirusTotal (87105) y ruido de integridad de archivos (550).

3. Cobertura de código

Métrica	Cubierto	Total	Cobertura	Umbral	Estado
Sentencias	181	184	98,4%	85%	**Cumple**
Ramas (branches)	75	82	91,5%	85%	**Cumple**
Funciones	31	32	96,9%	85%	**Cumple**
Líneas	175	177	98,9%	85%	**Cumple**

Por archivo

Archivo	Sentencias	Ramas	Funciones	Líneas
`src/app.js`	96,4%	82,3%	92,8%	97,6%
`src/auth.js`	100,0%	100,0%	100,0%	100,0%
`src/db.js`	100,0%	100,0%	100,0%	100,0%
`src/enrichment.js`	100,0%	100,0%	100,0%	100,0%
`src/normalize.js`	100,0%	87,5%	100,0%	100,0%
`src/wazuh.js`	100,0%	100,0%	100,0%	100,0%
Total	**98,4%**	**91,5%**	**96,9%**	**98,9%**

La medición abarca todo el código de la aplicación (`orchestrator/src/`). Quedan fuera los scripts operativos de uso único (`block_ip.js`, prueba de concepto de la fase 0, y `scripts/limpiar-ruido.js`). Las únicas líneas no ejecutadas corresponden al arranque del servidor HTTP (`app.listen`): las pruebas montan la aplicación en memoria sin abrir un puerto.

Aplicación del umbral en integración continua

Cada *pull request* hacia la rama principal ejecuta la suite completa en GitHub Actions. Si la cobertura de cualquiera de las cuatro métricas cae por debajo del 85%, el job falla y el cambio no debe integrarse. El resultado se publica como tabla en el resumen de la ejecución y el reporte HTML navegable (cobertura línea por línea) queda disponible como artefacto descargable. Se verificó que el mecanismo funciona forzando un umbral de 99,5%: la suite falló con `coverage threshold for lines (99.5%) not met`.

4. Catálogo de pruebas automatizadas

Cada grupo indica la parte del sistema que valida, el caso de uso al que pertenece y el riesgo que mitiga.

Ingesta y normalización · CU-01 · `webhook.test.js`

Riesgo mitigado: Una alerta de Wazuh mal formada o incompleta corrompe la base o tira el servicio.

Caso de prueba	Resultado
el webhook normaliza y persiste una alerta de brute-force SSH	**Aprobado**
el webhook rechaza una alerta con campos faltantes	**Aprobado**
GET /incidents devuelve la alerta persistida	**Aprobado**

Ingesta de malware · CU-02 · `webhookMalware.test.js`

Riesgo mitigado: Una alerta sin IP atacante (FIM/VirusTotal) se rechaza y el incidente se pierde. Defecto real encontrado el 02/10.

Caso de prueba	Resultado
el webhook acepta una alerta de VirusTotal/FIM (CU-02) sin srcIp	**Aprobado**
block-ip rechaza con 422 un incidente sin srcIp (ej. CU-02)	**Aprobado**

Filtro de ruido · CU-01 / CU-02 · `alertFilter.test.js`

Riesgo mitigado: Fatiga de alertas: 1.320 de 1.334 incidentes eran ruido (13/10).

Caso de prueba	Resultado
una alerta de FIM sobre archivos del sistema no crea incidente	**Aprobado**
una alerta ignorada igual se valida: si esta rota da 422	**Aprobado**
solo pasan las familias de los casos de uso	**Aprobado**

Respuesta: bloquear IP · CU-01 · `blockIp.test.js`

Riesgo mitigado: El incidente queda marcado como mitigado aunque Wazuh haya rechazado la acción.

Caso de prueba	Resultado
bloquea un incidente existente y lo marca como blocked	**Aprobado**
devuelve 404 si el incidente no existe	**Aprobado**
devuelve 502 si Wazuh rechaza el comando	**Aprobado**

Respuesta: aislar host · CU-02 · `isolateHost.test.js`

Riesgo mitigado: Idem, para el aislamiento del endpoint.

Caso de prueba	Resultado
aisla un incidente existente y lo marca como isolated	**Aprobado**
devuelve 404 si el incidente no existe	**Aprobado**
devuelve 502 si Wazuh rechaza el comando	**Aprobado**

Cliente de la API de Wazuh · CU-01 / CU-02 · `wazuh.test.js`

Riesgo mitigado: Wazuh responde HTTP 200 con la acción fallida en el cuerpo (error 1703, 04/08): la app informaría un bloqueo que no ocurrió.

Caso de prueba	Resultado
authenticate — devuelve el token JWT del manager	**Aprobado**
authenticate — credenciales rechazadas -> error con el status	**Aprobado**
blockIp — HTTP 200 sin items fallidos -> ok	**Aprobado**
blockIp — status HTTP de error -> rechaza	**Aprobado**
blockIp — HTTP 200 pero error en el body -> rechaza	**Aprobado**
blockIp — HTTP 200 pero el agente fallo (ej. 1703 en el agente 000) -> rechaza	**Aprobado**
isolateHost — HTTP 200 sin items fallidos -> ok	**Aprobado**
isolateHost — status HTTP de error -> rechaza	**Aprobado**
isolateHost — HTTP 200 pero error en el body -> rechaza	**Aprobado**
isolateHost — HTTP 200 pero el agente fallo (ej. 1703 en el agente 000) -> rechaza	**Aprobado**
ninguna llamada deja que axios tire por status HTTP: el status se evalua a mano	**Aprobado**
blockIp manda la IP en alert.data.srcip (lo que lee firewall-drop)	**Aprobado**
isolateHost manda !isolate-host sin IP (aisla al propio agente)	**Aprobado**

Enriquecimiento (VirusTotal / AbuseIPDB) · CU-01 · `enrichment.test.js`

Riesgo mitigado: Cuota gratuita agotada, red caída o API caída bloquean o rompen la ingesta.

Caso de prueba	Resultado
sin API keys configuradas, no pega a la red y devuelve null en ambas	**Aprobado**
con ambas API keys, consulta VT y AbuseIPDB y resume la respuesta	**Aprobado**
cachea el resultado: una segunda consulta a la misma IP no vuelve a pegarle a la red	**Aprobado**
si la red falla (timeout, DNS), no rompe: devuelve null para esa fuente	**Aprobado**
con una sola API key, consulta solo esa fuente	**Aprobado**
si la API responde con error, no rompe: devuelve null para esa fuente	**Aprobado**

Consulta y robustez · Transversal · `incidents.test.js`

Riesgo mitigado: Fallas de servicios auxiliares (enriquecimiento, auditoría) interrumpen el flujo principal.

Caso de prueba	Resultado
GET /incidents/:id devuelve el incidente	**Aprobado**
GET /incidents/:id inexistente -> 404	**Aprobado**
un token adulterado o vencido -> 401	**Aprobado**
el webhook guarda el enriquecimiento cuando VT/AbuseIPDB responden	**Aprobado**
sin datos de enriquecimiento, el incidente queda sin tocar	**Aprobado**
si el enriquecimiento falla, el webhook igual responde 201	**Aprobado**
si falla la auditoria, el login igual funciona (la auditoria no es bloqueante)	**Aprobado**

Autenticación (JWT) · Seguridad · `auth.test.js`

Riesgo mitigado: Acceso a incidentes o acciones sin credenciales válidas.

Caso de prueba	Resultado
login devuelve un token con credenciales validas	**Aprobado**
login rechaza credenciales invalidas	**Aprobado**
GET /incidents sin token devuelve 401	**Aprobado**

Perfil del analista · Soporte · `me.test.js`

Riesgo mitigado: Datos de perfil inválidos o accesibles sin autenticación.

Caso de prueba	Resultado
GET /me sin token devuelve 401	**Aprobado**
GET /me crea y devuelve el perfil del usuario autenticado	**Aprobado**
PUT /me actualiza nombre y email	**Aprobado**
PUT /me sin name/email devuelve 422	**Aprobado**

Auditoría · Seguridad · `audit.test.js`

Riesgo mitigado: Acciones del analista sin registro trazable.

Caso de prueba	Resultado
GET /audit sin token devuelve 401	**Aprobado**
un login exitoso queda registrado en el audit log	**Aprobado**

5. Pruebas de punta a punta contra infraestructura real

Ejecutadas sobre la infraestructura de AWS (manager Wazuh 4.8.2, endpoint víctima con agente Wazuh, orquestador) y la app instalada en un celular Android conectado por Tailscale. Registro completo en `progress.md`.

Fecha	Escenario	Resultado verificado
21/08	Bloqueo de IP vía API de Wazuh	Regla `DROP` creada en `iptables` del endpoint.
21/08	Primer flujo completo desde el celular	Login → incidente → step-up biométrico → bloqueo, por Tailscale. Puerto del orquestador no accesible desde Internet.
26/08	Fuerza bruta SSH real	Regla 5712 disparada; el incidente se crea automáticamente, sin intervención manual.
02/10	CU-02: archivo malicioso (EICAR)	FIM detecta el archivo → VirusTotal (65/75 motores) → alerta 87105 → incidente en el orquestador.
08/10	CU-02: aislar host desde la app	Endpoint sin conexiones nuevas; el agente Wazuh sigue activo (único canal permitido).
08/10	CU-01: bloquear IP desde la app	Conexión TCP del atacante al puerto 22 rechazada tras la acción.
08/10	Enriquecimiento en la app	Tarjeta de inteligencia de amenazas con datos reales de VirusTotal y AbuseIPDB.
13/10	Filtro de ruido	Alerta EICAR nueva llega como incidente; escaneo de integridad posterior no genera incidente. Base depurada de 1.335 a 15 incidentes reales.
13/10	Despliegue continuo	Merge a la rama principal → plan de Terraform sin cambios destructivos → despliegue automático del orquestador vía AWS SSM.

6. Defectos detectados por las pruebas

Fecha	Defecto	Cómo se detectó	Resolución
04/08	El manager (agente 000) rechaza acciones de respuesta (error 1703) devolviendo HTTP 200.	Prueba de concepto contra la API	Endpoint víctima dedicado; validación de `total_failed_items` (cubierta en `wazuh.test.js`).
04/08	El bloqueo informaba éxito sin crear la regla de firewall: la IP debía enviarse en `alert.data.srcip`.	Verificación en `iptables`	Cuerpo de la solicitud corregido (cubierto en `wazuh.test.js`).
02/10	Las alertas de malware se rechazaban porque la IP de origen era obligatoria.	Revisión de código con alerta simulada	IP opcional (cubierto en `webhookMalware.test.js`); confirmado horas después con la alerta real.
08/10	Aislar host informaba éxito sin aislar: el script estaba instalado con un nombre distinto al invocado.	Prueba de punta a punta	Instalación corregida y documentada.
13/10	1.320 de 1.334 incidentes eran ruido (actualizaciones del sistema).	Verificación posterior al despliegue	Filtro en el manager y en el orquestador (cubierto en `alertFilter.test.js`).
13/10	Pruebas dependientes de la configuración local (`.env`).	Primera ejecución en CI	Configuración de pruebas autocontenida.

7. Alcance y limitaciones

- **Aplicación móvil.** La cobertura automatizada abarca el backend. La app se valida con las pruebas de punta a punta de la sección 5 en un dispositivo real; incorporar pruebas automatizadas de la capa de

servicios de la app queda como trabajo futuro.

- **Frontera con Wazuh.** En las pruebas automatizadas, Wazuh se simula a nivel HTTP. Su comportamiento real (ejecución de la acción en el agente, efecto en el firewall) se valida en las pruebas de punta a punta.
- **Pruebas de punta a punta manuales.** Están documentadas con fecha y evidencia, pero no automatizadas; automatizarlas contra un entorno efímero es una mejora propuesta.